GYANMANJARI INNOVATIVE UNIVERSITY

Gyanmanjari Institute of Technology – Faculty of Engineering & Technology

Literature Review

Structured review of scholarly articles on randomized, balanced and self-adjusting search structures

Student Name	_____	Enrollment No.	_____
Subject	Research Methodology (METXX11501)	Semester	1 (M.Tech)
Activity	ALA 1	Marks	10

Research problem used in these activities: Comparative Search Efficiency of a Randomized Skip List, a Deterministic Red-Black Tree and a Self-Adjusting Splay Tree under Different Insertion Orders and Access Patterns.
(Illustrative problem selected for this submission; replace it with the faculty-assigned problem if a different one is given.)

1. Purpose and Scope

This activity reviews scholarly research relevant to the research problem: how the **skip list** (randomized), the **red-black tree** (deterministic, balanced) and the **splay tree** (self-adjusting) compare in search cost. The review is organised in a structured format: search strategy, a summary table (objective, method, findings, limitations), a concept matrix, a critical synthesis, the identified research gaps and the research objectives. It is a *focused* review of nine key articles (foundational and methodological), not an exhaustive survey of the whole field.

2. Literature Search Strategy

Item	Details
Sources searched	Google Scholar, ACM Digital Library, IEEE Xplore, SpringerLink, DBLP computer-science bibliography; publisher/author copies for verification.
Keywords	“skip list”, “red-black tree”, “splay tree”, “self-adjusting binary search tree”, “randomized search tree”, “B-tree”, “deterministic skip list”, “search cost analysis”, “performance evaluation methodology”.
Search technique	Boolean combinations (e.g. skip list AND (analysis OR performance)), then backward citation chasing from Pugh (1990) and forward chasing through papers that cite it.
Inclusion criteria	Peer-reviewed conference/journal papers; introduces, analyses or evaluates a search structure or evaluation methodology; full text and bibliographic details verified.
Exclusion criteria	Blog posts, unreviewed web articles, papers on unrelated structures, duplicates.
How papers were read	Abstract and introduction first; then method and results; key ideas and limitations noted in a standard template (Section 3).

3. Structured Summary of Reviewed Articles

ID	Reference	Objective	Method	Key findings	Limitations / relevance
P1	Bayer & McCreight (1972)	Index large ordered files stored in pages (disk blocks) with guaranteed fast access and update.	Defines the B-tree, gives search/insert/delete algorithms and analyses cost and storage use.	Access, insertion and deletion take time logarithmic in file size; nodes (pages) stay at least about half full.	Designed for paged external memory. Foundation of multiway balanced trees; related to the 2-3 tree/B-tree compared in Course ADS.
P2	Guibas & Sedgewick (1978)	Give a single framework for several balanced-tree schemes.	Represents balanced trees with red/black node colours and rotations; shows how 2-3 and 2-3-4 trees map to it.	Red-black trees give worst-case logarithmic search and update with simple local rebalancing.	Conceptual/theoretical paper. Source of the **deterministic balanced** baseline (red-black tree).
P3	Sleator & Tarjan (1985)	Design a binary search tree that adapts to the sequence of accesses without storing balance data.	Introduces splay trees (move accessed node to the root by rotations); amortized analysis with a potential function.	Every operation takes O(log n) amortized time; the tree is competitive with the best static tree for access sequences with skew.	Bounds are amortized, not per-operation; searches modify the tree. Source of the **self-adjusting** comparison structure.
P4	Aragon & Seidel (1989)	Obtain balanced search trees using randomization.	Randomized search trees (treaps): each key gets a random priority; the tree is a heap on priorities.	Expected O(log n) time for search, insert and delete; shape does not depend on the insertion order.	Guarantee is in expectation. Shows the general idea of **randomized balancing** that skip lists share.
P5	Pugh (1990)	Provide a simple alternative to balanced trees.	Introduces skip lists: nodes get random heights; algorithms for search, insert, delete; probabilistic analysis and timing comparisons with other structures.	Expected O(log n) operations; the author reports that skip lists are simpler and, in his tests, competitive in speed and space with balanced and self-adjusting trees.	Probabilistic guarantee only; analysis gives upper bounds; timings relate to the hardware, language and implementations of that time.
P6	Munro, Papadakis & Sedgewick (1992)	Obtain skip-list-like structures with guaranteed (worst-case) logarithmic cost.	Deterministic skip lists: the number of nodes between taller nodes is kept within fixed limits.	Guaranteed logarithmic search, insert and delete; competitive in time and space with balanced trees and, the authors argue, simpler.	More complex than the random version. Provides the **deterministic counterpart** of the skip list.
P7	Papadakis, Munro & Poblete (1992)	Analyse the expected search and update cost of Pugh's skip list precisely.	Mathematical analysis giving exact and asymptotic expected search cost (fixed key and average key) and insert/delete costs.	Pugh's upper bound on expected search cost is fairly tight for the cases of interest.	Theoretical; no implementation study. Gives the **theoretical benchmark** (about $\log_2 n$ / p comparisons) for our measurements.
P8	Herlihy, Lev, Luchangco & Shavit (2007)	Design a simple, scalable concurrent skip list.	Optimistic lock-based algorithm (searching without locks, short validation); experiments on multiprocessor hardware.	Performs as well as the best known lock-free skip list (Lea's) under the most common usage patterns.	Concerned with multi-threading, not single-thread search cost; shows the practical interest of skip lists.
P9	Georges, Buytaert & Eeckhout (2007)	Show how to evaluate performance without misleading conclusions.	Surveys common practice in Java benchmarking and proposes statistically rigorous data analysis (repeated runs, confidence intervals).	Non-rigorous reporting can lead to wrong conclusions; replication and confidence intervals are needed.	Written for Java/JVM non-determinism. Supplies the **methodological guidance** for our replicated experiment.

4. Concept Matrix (key ideas across papers)

Paper	Balanced (worst-case)	Self-adjusting	Random-ized	Analysis	Experi-ments	Statistical method
P1	X			X		
P2	X			X		
P3		X		X		
P4			X	X		
P5			X	X	X	
P6	X			X		
P7			X	X		
P8			X		X	
P9					X	X

X = the theme is a main concern of the paper. P1, P2 and P6 all pursue deterministic (worst-case) balance; among the reviewed papers, P5, P8 and P9 place the most emphasis on experimental evaluation.

5. Critical Review and Synthesis

5.1 Three design philosophies for fast search

The literature shows three ways to keep a search structure fast. **Deterministic balancing** (P1, P2, P6) enforces structural invariants and gives worst-case guarantees at the cost of intricate insert/delete cases. **Self-adjusting** trees (P3) store no balance data and rely on amortization, so a single operation may be slow but long sequences are efficient; they also adapt to skewed access. **Randomized balancing** (P4, P5) replaces rules by coin tosses and gives simple code with expected O(log n) cost independent of insertion order.

5.2 Strengths of the existing work

- Strong theory: tight, mathematically proven bounds for each structure (P1–P4, P6, P7).
- The author of the skip list reports (P5) competitive practical performance, and later work shows skip lists are practical in concurrent settings (P8).
- Methodological work (P9) provides a rigorous basis for experimental comparison.

5.3 Weaknesses and limitations noted

- Guarantees differ in type (worst-case, amortized, expected), so they cannot be compared directly from theory alone.
- The experiments in the foundational papers were done on hardware, languages and implementations of their time (P5), and may not transfer to present-day settings.
- Experimental comparisons in the reviewed articles do not, in general, combine *several insertion orders and several access patterns* with *replicated runs and formal hypothesis tests*.
- Exact analyses (P7) describe the skip list alone and do not compare it with other structures under skewed workloads.

6. Research Gaps Identified

Gap	Evidence from the review	How the study addresses it
G1: Few replicated, statistically tested comparisons of randomized, deterministic and self-adjusting structures	P5 reports timings; P3, P6, P7 are mainly analytical; P9 shows rigorous methods are often missing	30 independent replications per condition, Welch t-tests, ANOVA, confidence intervals and effect sizes
G2: Interaction of *insertion order* and *access pattern* is rarely studied together	P4, P5 claim order-independence for randomized structures; P3 stresses adaptivity to access patterns	2 insertion orders × 2 access patterns × 3 structures factorial design
G3: Theory (expected cost) is seldom checked against measurements of a simple modern implementation	P7 gives exact expected cost	Measured cost compared with the theoretical value $2 \cdot \log_2 n$

7. Research Problem and Objectives Derived from the Review

Problem statement: It is not clear how much search efficiency a student or practitioner gains or loses by choosing a randomized skip list instead of a deterministic red-black tree or a self-adjusting splay tree, and whether this depends on the insertion order and on whether queries are uniform or skewed.

Research objectives:

- O1: To measure the average search cost (nodes inspected per search) of the three structures under uniform queries.
- O2: To test whether a splay tree outperforms a red-black tree when queries are skewed towards a few keys.
- O3: To test whether the randomized skip list is insensitive to insertion order, in contrast to the splay tree.
- O4: To compare the measured skip-list cost with the theoretical expected cost and check that the random levels follow the geometric distribution.

8. References (as cited above)

- Bayer, R. and McCreight, E. (1972). Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3), 173–189.
- Guibas, L. J. and Sedgewick, R. (1978). A dichromatic framework for balanced trees. *Proc. 19th Annual Symposium on Foundations of Computer Science (FOCS)*, 8–21.
- Sleator, D. D. and Tarjan, R. E. (1985). Self-adjusting binary search trees. *Journal of the ACM*, 32(3), 652–686.
- Aragon, C. R. and Seidel, R. G. (1989). Randomized search trees. *Proc. 30th Annual Symposium on Foundations of Computer Science (FOCS)*, 540–545.
- Pugh, W. (1990). Skip lists: a probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6), 668–676.
- Munro, J. I., Papadakis, T. and Sedgewick, R. (1992). Deterministic skip lists. *Proc. 3rd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 367–375.
- Papadakis, T., Munro, J. I. and Poblete, P. V. (1992). Average search and update costs in skip lists. *BIT*, 32(2), 316–332.

- Herlihy, M., Lev, Y., Luchangco, V. and Shavit, N. (2007). A simple optimistic skiplist algorithm. *Structural Information and Communication Complexity (SIROCCO 2007)*, LNCS 4474, Springer.
- Georges, A., Buytaert, D. and Eeckhout, L. (2007). Statistically rigorous Java performance evaluation. *Proc. OOPSLA '07 (ACM SIGPLAN)*, 57–76.

Note: bibliographic details were checked against publisher or bibliography records; page ranges may be cross-checked on the publisher page before final submission.